

Revisión de mejoras del Trabajo Terminal

TT2026-2_IA05

Sistema de detección y seguimiento de objetos
desde perspectiva aérea con UAV

Miguel Alejandro Flores Sotelo
Sergio de Jesús Castillo Molano

Generado el 2026-06-03

Índice de avance

#	Área	Estado
1	Novedad / aporte del proyecto	Redactado
2	Métricas adicionales	Redactado
3	Desbalance de clases	Redactado
4	Justificación de dos personas + elección de modelos	Redactado
5	Marco de experimentación justo	Redactado
6	Mejora de resultados	Redactado
A	Anexo: alternativas a Google Colab	Redactado

1. Novedad y aporte del proyecto

1.1 Marco: qué constituye un aporte en un Trabajo Terminal

Un Trabajo Terminal de ingeniería no exige una contribución de investigación original en el sentido de proponer una arquitectura nueva o superar el estado del arte. El aporte válido puede ser de naturaleza ingenieril e integrativa: resolver un problema aplicado con rigor, integrar componentes que en la literatura suelen estudiarse por separado, hacerlo bajo restricciones reales (hardware limitado, vuelo real) y documentarlo de forma reproducible. Esta distinción condiciona cómo se enuncia y se defiende la contribución ante el jurado.

1.2 Diagnóstico del estado del arte

La revisión de literatura confirma que las piezas individuales del proyecto ya existen, por lo que la contribución no puede sostenerse sobre ninguna de ellas de forma aislada.

Componente	Situación en la literatura
Comparación CNN vs Transformer en VisDrone	Muy saturada. Existe una familia de detectores transformer para UAV (UAV-DETR, RT-DETR++, SF-DETR, EABI-DETR), además de comparativas directas RT-DETR vs YOLO.
Despliegue en NVIDIA Jetson	Terreno trillado. Existen trabajos de benchmarking de YOLO en Jetson Orin y guías de despliegue con TensorRT.
Detección de objetos pequeños en imagen aérea	Campo extenso (tiling/SAHI, BiFPN, módulos de atención, cabezas de alta resolución).

Componente	Situación en la literatura
Validación cruzada VisDrone-UAVDT	Existe, pero abordada con métodos pesados (desentrenado de dominio, pre-entrenamiento sintético). Una comparación directa y limpia sigue siendo un ángulo menos saturado.

Conclusión: la hipótesis inicial del proyecto ("comparar CNN vs Transformer y desplegar en hardware limitado") es insuficiente como aporte si se enuncia de forma plana, porque cada parte por separado ya está publicada.

Cifras de referencia de la literatura

Estas cifras sitúan los resultados propios y se reutilizan en el área 6.

Modelo / trabajo	Resultado en VisDrone
YOLOv8-M (modelo mediano)	aprox. 24.6 % AP (mAP50-95)
RT-DETR (baseline)	aprox. 48 % mAP@0.5
LAF-YOLOv10 (especializado)	35.1 % mAP@0.5; 24.3 FPS en Jetson Orin Nano (FP16)
YOLOv8n del proyecto (imgsz=1280)	47.3 % mAP@0.5; 28.6 % mAP50-95

Observación relevante: el modelo nano del proyecto, entrenado a 1280 px, alcanza un mAP50-95 superior al YOLOv8-M reportado y un mAP@0.5 cercano al baseline de RT-DETR. Los resultados propios son competitivos, no deficientes; este punto se desarrolla en el área 6.

1.3 Pregunta de investigación

Pregunta general. ¿Qué arquitectura de detección —una CNN pura (YOLOv8n) frente a una híbrida CNN-Transformer (RT-DETR)— ofrece el mejor compromiso entre precisión, eficiencia computacional y capacidad de generalización para la detección y seguimiento de objetos en tiempo real desde un UAV con cómputo de borde restringido (NVIDIA Jetson Orin Nano 8 GB)?

Sub-preguntas.

1. Precisión: ¿qué diferencias de mAP y F1 por clase presentan ambas arquitecturas en VisDrone, en particular en clases minoritarias y objetos pequeños?
2. Eficiencia: ¿cuál ofrece mejor FPS, latencia, FLOPs y uso de memoria (VRAM/RAM) en el hardware real de despliegue?
3. Generalización: ¿cuál degrada menos ante cambio de dominio (entrenado en VisDrone, evaluado en UAVDT)?
4. Sistema: ¿el pipeline integrado (detección, seguimiento y georreferenciación) alcanza tiempo real (igual o mayor a 20 FPS) en la Jetson manteniendo precisión utilizable?

Hipótesis.

- H1. RT-DETR ofrecerá mayor precisión pero menor FPS y mayor uso de recursos; YOLOv8n, lo inverso. El punto de equilibrio determinará la recomendación para UAV.
- H2. Ambas arquitecturas degradarán al pasar a UAVDT, pero no por igual; la magnitud de la caída indicará su robustez de dominio.
- H3. Probablemente solo una de las dos (previsiblemente YOLOv8n) sostendrá 20 FPS o más en la Jetson, lo que condicionará la elección de despliegue con independencia de quién tenga mayor mAP.

1.4 Pilares del aporte

La contribución se articula en tres pilares, desplazando el peso del comparativo (débil por sí solo) hacia la caracterización en borde y el sistema integral.

Estudio comparativo controlado (CNN vs Transformer). No como mera comparación, sino como comparación justa y reproducible bajo condiciones idénticas, orientada a un objetivo de despliegue en borde. Es soporte metodológico, no el titular.

Caracterización del compromiso precisión-eficiencia en un objetivo de borde concreto. Construcción de la frontera de Pareto (mAP frente a FPS, latencia y uso de memoria) sobre el hardware real del UAV. Responde una pregunta de ingeniería con valor práctico: para un UAV con cómputo limitado, qué modelo conviene y por qué.

Sistema integral de extremo a extremo con vuelo real y georreferenciación. El lazo completo: detección, seguimiento (ByteTrack/DeepSORT), despliegue embebido (TensorRT FP16 en Jetson), streaming RTSP y georreferenciación mediante fusión con la telemetría del controlador de vuelo Pixhawk. El paso de coordenada en píxeles a coordenada en el mundo, usando la pose del UAV, constituye una contribución de integración de sistemas poco frecuente a nivel de Trabajo Terminal.

1.5 Refuerzos del aporte

- **Validación cruzada de dominio (VisDrone a UAVDT).** Mide la generalización ante cambio de dominio, algo que la mayoría de los benchmarks de un solo conjunto no evalúa. Requiere documentar el mapeo de clases, pues UAVDT contempla tres clases (car, truck, bus) frente a las diez de VisDrone.
- **Pregunta de investigación explícita.** Convierte las seis áreas en metodología para responderla, haciendo evidente la contribución.
- **Medición en hardware real, no en GPU de escritorio.** Reportar FPS y uso de memoria sobre la propia Jetson es precisamente lo que falta en muchos trabajos; la restricción de hardware se convierte en fortaleza.

1.6 Texto formal de aporte (borrador para protocolo/tesis)

El presente Trabajo Terminal aporta un sistema integral de detección y seguimiento de objetos desde perspectiva aérea desplegable en cómputo de borde, junto con una caracterización experimental rigurosa y reproducible del compromiso precisión-eficiencia-generalización entre una arquitectura convolucional (YOLOv8n) y una híbrida convolución-transformador (RT-DETR). A diferencia de los estudios comparativos existentes, evaluados típicamente sobre un único conjunto de datos y en hardware de escritorio, esta comparación se realiza bajo condiciones experimentales controladas, con validación cruzada de dominio (VisDrone a UAVDT) y medición directa sobre el hardware de despliegue final (NVIDIA Jetson Orin Nano 8 GB montada sobre un UAV F450), cerrando el ciclo hasta la georreferenciación de las detecciones mediante fusión con la telemetría del controlador de vuelo Pixhawk. El resultado es un criterio de selección de arquitectura fundamentado empíricamente para aplicaciones de detección aérea en tiempo real con recursos limitados.

1.7 Cómo enunciarlo ante el jurado

Evitar formulaciones planas del tipo "el aporte es comparar dos modelos y ponerlos en una Jetson", pues invitan a la objeción de que cada parte ya existe. Enunciar la contribución como sistema integral, caracterización rigurosa, condiciones controladas, validación cruzada de dominio, medición en hardware real y georreferenciación; cada uno de estos términos es verificable y distingue el trabajo del benchmark genérico.

1.8 Plan de validación comparativa (tras el reentrenamiento)

Una vez reentrenados los modelos (YOLOv8n y RT-DETR) y obtenidos los resultados definitivos, se compararán las métricas propias contra trabajos publicados que apliquen las mismas arquitecturas sobre VisDrone o escenarios aéreos equivalentes. El objetivo es demostrar de forma verificable que los resultados son competitivos con el estado del arte y no deficientes. Cada cifra de referencia se acompañará de su fuente citada y verificable, de modo que el dato sea comprobable ante el jurado. Las cifras base ya recogidas en la sección 1.2 constituyen el punto de partida de esta comparación.

Bibliografía consultada (área 1)

1. Ultralytics. RTDETRv2 vs YOLOv8: A Technical Comparison. <https://docs.ultralytics.com/compare/rtdetr-vs-yolov8/>
2. UAV-DETR: Efficient End-to-End Object Detection for UAV Imagery. arXiv 2501.01855. <https://arxiv.org/html/2501.01855v1>
3. EABI-DETR: An Efficient Aerial Small Object Detection Network. PMC12650067. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC12650067/>
4. LAF-YOLOv10 (despliegue en Jetson Orin Nano, 24.3 FPS FP16). arXiv 2602.13378. <https://arxiv.org/pdf/2602.13378>
5. Benchmarking YOLOv8 Variants on Jetson Orin NX. MDPI Computers 15(2):74. <https://www.mdpi.com/2073-431X/15/2/74>
6. Domain Feature Decomposition for Efficient Object Detection in Aerial Images (VisDrone/UAVDT). MDPI Remote Sensing 16(9):1626. <https://www.mdpi.com/2072-4292/16/9/1626>

2. Métricas adicionales

2.1 Motivación

Las métricas de precisión (mAP, precisión, recall) describen qué tan bien detecta el modelo, pero no qué tan viable es montarlo en un UAV. Para justificar el despliegue se requieren métricas de eficiencia (velocidad) y de costo computacional (cómputo y memoria). Solo el conjunto completo permite responder la sub-pregunta 2 y construir la frontera de Pareto descrita en el área 1.

2.2 Clasificación de las métricas según su origen

Métrica	Qué mide	Origen	
mAP@0.5, mAP@0.5:0.95	Precisión de detección	Ultralytics	directo
Precisión, Recall	Falsos positivos / falsos negativos	Ultralytics	directo
F1-score (global y por clase)	Equilibrio precisión-recall	Ultralytics	directo
Parámetros (M)	Tamaño del modelo	Ultralytics (model.info)	directo
FLOPs (GFLOPs)	Costo de cómputo teórico	Ultralytics (model.info)	directo
Latencia (pre/inf/post, ms)	Tiempo de proceso por imagen	Ultralytics (results.speed)	directo
FPS	Cuadros por segundo	Derivada de la latencia	

Métrica	Qué mide	Origen
VRAM (MB)	Memoria GPU en inferencia	Código adicional (torch.cuda)
RAM (MB)	Memoria de sistema	Código adicional (psutil)

Conviene distinguir tres grupos según qué miden y cuándo se obtienen:

- Métricas de precisión (mAP, precisión, recall, F1). Las calcula Ultralytics de forma automática en la validación que ejecuta al final de cada época durante el entrenamiento, y son reproducibles en cualquier momento con `model.val()`. Su valor no depende del hardware: el mAP es el mismo en cualquier GPU. Se obtienen al (re)entrenar. Nota: el F1 lo calcula Ultralytics (curva BoxF1), pero el script de entrenamiento actual solo persiste mAP, precisión y recall; debe añadirse $F1 = 2PR/(P+R)$ al guardado de métricas.
- Complejidad del modelo (parámetros, FLOPs). Son propiedades fijas del modelo que Ultralytics reporta; iguales en cualquier equipo. Los FLOPs (operaciones de punto flotante por imagen) miden el costo teórico de cómputo, distinto de los FLOPS (operaciones por segundo) que caracterizan la potencia de una GPU.
- Eficiencia dependiente del hardware (latencia, FPS, VRAM, RAM). Se miden ejecutando el modelo ya entrenado en inferencia sobre el equipo objetivo. Aquí el valor sí cambia con el hardware, por lo que deben medirse en la laptop y en la Jetson, nunca en la GPU rentada que se haya usado para entrenar.

2.3 Relevancia de cada métrica para el despliegue en UAV

- F1-score. En aplicaciones de vigilancia o seguridad desde UAV, un falso negativo puede ser más costoso que un falso positivo. El F1 resume ese equilibrio y, calculado por clase, revela en qué categorías el modelo falla de forma crítica.
- FLOPs y parámetros. Determinan si el modelo cabe y se ejecuta en cómputo embebido. Sustentan el argumento de que un modelo nano puede ser preferible a uno más preciso pero más pesado.
- Latencia y FPS. La literatura cita un umbral de 20 FPS como mínimo para operación en vuelo; por debajo de ese valor el seguimiento se degrada. Es un criterio de viabilidad, no un complemento.
- VRAM y RAM. La Jetson Orin Nano dispone de 8 GB compartidos entre CPU y GPU. Si el modelo junto con el resto del pipeline (seguimiento y streaming) no cabe en memoria, el sistema no es desplegable.

2.4 Obtención de cada métrica (Ultralytics 8.4.37)

Precisión, F1, latencia y FPS desde la validación:

```
from ultralytics import YOLO

model = YOLO("runs/yolov8n/.../weights/best.pt")
m = model.val(data="configs/visdrone.yaml", imgsz=1280)

mAP50      = m.box.map50          # mAP@0.5
mAP50_95   = m.box.map           # mAP@0.5:0.95
P, R       = m.box.mp, m.box.mr  # precision y recall medios
F1         = 2 * P * R / (P + R) # F1 global
F1_clase   = m.box.f1            # F1 por clase (arreglo)

inf_ms     = m.speed["inference"] # ms por imagen (inferencia)
fps        = 1000 / inf_ms         # FPS
```

FLOPs y parámetros:

```
# model.info() devuelve (capas, parametros, gradientes, GFLOPs)
```

```
_, n_params, _, gflops = model.info()
print(f"{n_params/1e6:.2f} M parametros, {gflops:.1f} GFLOPs")
```

VRAM y RAM:

```
import torch, psutil, os
torch.cuda.reset_peak_memory_stats()
model.val(data="configs/visdrone.yaml", imgsz=1280)
vram_mb = torch.cuda.max_memory_allocated() / 1024**2 # pico de VRAM
ram_mb = psutil.Process(os.getpid()).memory_info().rss / 1024**2
```

2.5 Dependencia del hardware y momento de medición (enlace con áreas 5 y 6)

- FLOPs, parametros y latencia teorica son propiedad del modelo y la resolucion; se miden una sola vez por modelo.
- FPS, VRAM y RAM dependen del hardware; deben reportarse dos veces: en la laptop (RTX 4050) y en la Jetson Orin Nano. La cifra valida para el caso de uso en UAV es siempre la medida en la Jetson.
- Distincion clave entre entrenamiento e inferencia. El entrenamiento puede realizarse en servicios externos en la nube (por su mayor capacidad y estabilidad), pero las metricas de eficiencia NO se miden alli: hacerlo mediria la GPU rentada, ajena al hardware del UAV. Dichas metricas se obtienen en la fase de inferencia, ejecutando el modelo entrenado sobre la laptop y sobre la Jetson. Entrenamiento y medicion de eficiencia son etapas distintas y se realizan en equipos distintos.
- La instrumentacion de estas metricas se integra en la fase final de evaluacion y mejora de resultados (area 6), una vez disponibles los pesos definitivos de ambos modelos.

2.6 Tabla comparativa final propuesta

Plantilla a completar con ambos modelos y ambos equipos de referencia.

Metrica	YOLOv8n	RT-DETR	Hardware
mAP@0.5			independiente
mAP@0.5:0.95			independiente
Precision			independiente
Recall			independiente
F1-score			independiente
Parametros (M)			independiente
FLOPs (GFLOPs)			independiente
Latencia inferencia (ms)			laptop / Jetson
FPS			laptop / Jetson
VRAM pico (MB)			laptop / Jetson
RAM (MB)			laptop / Jetson

3. Desbalance de clases

3.1 Magnitud del desbalance en VisDrone

Distribución de detecciones en el conjunto de entrenamiento (fuente: EDA propio,

results/tables/eda/01_distribucion_clases.csv).

Clase	Detecciones	Porcentaje
car	144,865	42.21
pedestrian	79,335	23.12
motor	29,642	8.64
people	27,059	7.88
van	24,950	7.27
truck	12,871	3.75
bicycle	10,477	3.05
bus	5,926	1.73
tricycle	4,803	1.40
awning-tricycle	3,243	0.95

La clase car tiene aproximadamente 45 veces más ejemplos que awning-tricycle. El desbalance es severo y se corresponde con los resultados del entrenamiento: car alcanza el mejor AP (0.853) y awning-tricycle el peor (0.191).

3.2 Efecto de las técnicas de augmentación configuradas sobre el balance

- Mosaic (1.0). Combina cuatro imágenes en una sola, aumentando la densidad de objetos, el contexto y la variación de escala. Selecciona las imágenes de forma aleatoria, por lo que en promedio conserva las proporciones de clases. Mejora la generalización pero no rebalancea.
- Mixup (0.15). Mezcla dos imágenes y sus etiquetas con transparencia. También muestrea de forma aleatoria, de modo que no altera el balance de clases.
- Copy_paste (0.3). Según la documentación de Ultralytics, esta augmentación opera sobre máscaras de segmentación: recorta objetos a partir de sus máscaras y los pega en otras imágenes. VisDrone se entrena aquí en modo detección, con cajas delimitadoras y sin máscaras, por lo que copy_paste no produce el efecto esperado. Además, aunque se ejecutara, no selecciona clases minoritarias de forma preferente, sino objetos de forma indistinta.

3.3 Conclusión

Ninguna de las tres técnicas rebalancea las clases minoritarias. Mosaic y mixup aportan robustez general y conviene mantenerlas, pero el desbalance permanece. El comentario del script que atribuye a copy_paste la copia de objetos minoritarios es incorrecto: ni selecciona minoritarios ni resulta efectivo en una tarea de detección sin máscaras.

3.4 Opciones para tratar el desbalance

Ultralytics no ofrece un parámetro nativo de pesos por clase en `model.train()`. Las técnicas disponibles se agrupan en dos niveles según dónde actúan.

Nivel de datos (se aplican antes del entrenamiento, sin modificar el modelo): - Oversampling. Hacer que las imágenes con clases raras aparezcan con mayor frecuencia durante el entrenamiento, por ejemplo duplicándolas en el conjunto de entrenamiento. - Augmentación dirigida. Aplicar aumentos adicionales únicamente sobre las imágenes que contienen clases raras.

Nivel de pérdida (modifican cómo el modelo penaliza los errores, objeto por objeto): - Pérdida ponderada por clase. Asignar a cada clase un peso inversamente proporcional a su frecuencia, de modo que equivocarse en una clase rara penalice más. - Focal loss. Concentrar el aprendizaje en los ejemplos difíciles y raros (se detalla en la sección 3.5).

La diferencia entre ambos niveles es relevante para el diseño experimental. Las técnicas de nivel de datos se aplican por igual a YOLOv8 y a RT-DETR, por lo que constituyen una intervención común a ambos modelos. Las de nivel de pérdida forman parte de la configuración interna de cada arquitectura: por defecto, YOLOv8 emplea BCE y RT-DETR emplea Varifocal/Focal Loss (ver sección 3.5). En este trabajo cada modelo se evalúa en su configuración por defecto, de modo que esa diferencia de pérdida se describe y analiza como una característica propia de cada arquitectura, no como algo que se modifique manualmente.

Aclaración: los hiperparámetros box, cls y dfl ponderan los componentes de la función de pérdida (caja, clasificación y DFL), no las clases individuales; subir cls afecta a toda la clasificación por igual y no corrige el desbalance.

3.5 Por qué RT-DETR usa focal loss por defecto y YOLOv8 no

Ambos modelos se entrenan a través de Ultralytics, pero su función de pérdida de clasificación difiere por diseño de cada arquitectura, no por la herramienta.

Qué es la BCE (entropía cruzada binaria), la pérdida que usa YOLOv8. En detección, YOLOv8 plantea la clasificación como un conjunto de decisiones binarias independientes: para cada predicción y cada clase responde a la pregunta de si esa clase está presente o no. La BCE compara la probabilidad que predice el modelo con la etiqueta real (1 si la clase está, 0 si no), penaliza las predicciones seguras y equivocadas y premia las seguras y correctas. Su característica central es que trata todos los ejemplos con el mismo peso.

Por qué la BCE sufre con el desbalance. Al ponderar por igual cada ejemplo, la pérdida total queda dominada por las clases abundantes y por la enorme cantidad de ejemplos fáciles (fondo y objetos comunes). El gradiente se orienta entonces sobre todo a acertar en las clases mayoritarias, mientras que las minoritarias apenas influyen en el aprendizaje. La BCE no dispone de ningún mecanismo para concentrarse en los ejemplos raros o difíciles, de ahí su dificultad ante datasets desbalanceados como VisDrone.

Qué es la focal loss. Es una variante de la entropía cruzada pensada para escenarios con fuerte desbalance. Introduce un factor de modulación $(1 - p)^\gamma$ que reduce de forma automática la contribución de los ejemplos fáciles (aquellos que el modelo ya clasifica con alta confianza) y mantiene el peso de los difíciles. De este modo el entrenamiento deja de estar dominado por la gran cantidad de ejemplos fáciles o frecuentes y se concentra en los casos difíciles, que suelen coincidir con los objetos raros, pequeños u ocluidos. En esencia, la focal loss es la BCE multiplicada por ese factor de modulación; por eso la clase FocalLoss de Ultralytics se construye envolviendo a la BCE.

- YOLOv8 utiliza entropía cruzada binaria (BCE) para la clasificación, junto con DFL y CloU para las cajas, sobre una asignación de etiquetas tipo task-aligned (uno-a-muchos). No emplea focal loss por defecto.
- RT-DETR, como miembro de la familia DETR, utiliza por defecto pérdida focal sobre un emparejamiento uno-a-uno (matching húngaro). En la implementación de Ultralytics 8.4.37, el modelo instancia su criterio con `use_vfl=True` (nn/tasks.py), por lo que aplica Varifocal Loss cuando hay objetos presentes y Focal Loss como respaldo; en ningún caso usa BCE simple para la clasificación. Verificado en el código instalado.

Por qué la familia DETR necesita focal loss: estos detectores generan un conjunto fijo de consultas (queries), de las cuales la gran mayoría corresponde al fondo. Esto produce un desbalance extremo entre primer plano y fondo. La focal loss reduce el peso de los ejemplos fáciles y abundantes (el fondo) y concentra el aprendizaje en los ejemplos difíciles y escasos, evitando que el fondo domine el gradiente.

Por qué YOLOv8 no la usa: su esquema de asignación de etiquetas y su diseño ya gestionan ese desbalance de otra manera, y empíricamente la BCE rinde bien con esa asignación. Es una decisión de diseño, no una carencia.

En este trabajo, cada arquitectura se evalúa en su configuración por defecto: YOLOv8 con BCE y RT-DETR con Varifocal/Focal Loss. Aunque técnicamente podría habilitarse focal loss en YOLOv8 (la clase FocalLoss ya existe en Ultralytics y bastaría una modificación localizada de su pérdida), se opta por comparar ambas arquitecturas tal como vienen, de modo que la función de pérdida se trata como una característica propia de cada una.

Cómo se verificó. Estas afirmaciones no se basan en la documentación general, sino en la inspección directa del código instalado de Ultralytics 8.4.37: en `v8DetectionLoss` la clasificación emplea `nn.BCEWithLogitsLoss` (`utils/loss.py`), mientras que RT-DETR instancia `RTDETRDetectionLoss` con `use_vfl=True` (`nn/tasks.py`), cuya rutina de pérdida aplica Varifocal o Focal Loss y nunca BCE simple. Se confirmó además en tiempo de ejecución.

Relevancia para el desbalance de clases: la focal loss mitiga de forma intrínseca el desbalance entre ejemplos frecuentes y raros. En consecuencia, RT-DETR dispone de un mecanismo nativo que compensa parcialmente el desbalance, mientras que la BCE de YOLOv8 trata todos los ejemplos por igual. Esta diferencia constituye un eje de análisis relevante para la comparación CNN frente a Transformer del trabajo.

3.6 Cómo se aplica el oversampling y su limitación en VisDrone

El oversampling actúa a nivel de datos: consiste en hacer que las imágenes que contienen clases raras aparezcan con mayor frecuencia durante el entrenamiento, por ejemplo duplicándolas dentro del conjunto de entrenamiento (replicando los pares imagen-etiqueta o repitiendo sus rutas en la lista de entrenamiento). No requiere modificar el modelo y se aplica igual a ambas arquitecturas.

Limitación en VisDrone. El oversampling tiene una restricción de fondo: la unidad de muestreo es la imagen completa, no el objeto individual. Durante el entrenamiento se pasa cada imagen con todas sus etiquetas a la vez, sin posibilidad de seleccionar únicamente los objetos de una clase. Como las imágenes de VisDrone son densas y multiclase, una imagen que contiene un `awning-tricycle` contiene además numerosos `car`, `pedestrian` y otros objetos. En consecuencia, al repetir esas imágenes para aumentar la presencia de la clase minoritaria, se repiten por igual todos los demás objetos que las acompañan. El efecto neto es que la proporción entre clases apenas cambia, porque las clases dominantes crecen en paralelo a las raras; el desbalance se corrige solo de forma parcial y, además, se incrementa el riesgo de sobreajuste sobre el reducido conjunto de imágenes duplicadas. Un control más fino solo se lograría a nivel de objeto (pérdida ponderada por clase o focal loss); en la comparación principal, no obstante, cada modelo se usa en su configuración por defecto (ver sección 3.5).

3.7 Decisión: el desbalance como eje de análisis, no como problema a forzar

A partir de las dos limitaciones anteriores, la decisión metodológica del trabajo es no aplicar un rebalanceo artificial en la comparación principal, sino tratar el desbalance como un eje de análisis. El razonamiento es el siguiente:

- El oversampling a nivel de imagen no permite rebalancear de forma limpia las clases minoritarias en VisDrone, por su densidad y carácter multiclase (sección 3.6).
- Los métodos a nivel de objeto (pérdida ponderada o focal loss) corregirían el desbalance, pero modificar la pérdida de YOLOv8 implicaría apartarse de su configuración por defecto, cuando la decisión es comparar ambas arquitecturas tal como vienen (sección 3.5).

En consecuencia, en lugar de forzar una técnica de balanceo de efecto dudoso, el trabajo:

- Reporta AP y F1 por clase, no solo el mAP global, para documentar el desbalance con transparencia y mostrar el desempeño real en las clases minoritarias.

- Convierte el desbalance en una pregunta de la propia comparación CNN frente a Transformer: evaluar si la focal loss nativa de RT-DETR maneja mejor las clases raras (awning-tricycle, bicycle, tricycle) que la BCE de YOLOv8.

De este modo, una limitación del dataset se transforma en una contribución analítica del trabajo, y se evita la incoherencia de aplicar una herramienta que se ha justificado como poco efectiva en este escenario.

Matiz honesto. Parte del bajo rendimiento en awning-tricycle y bicycle no se debe solo al desbalance, sino también a que son objetos pequeños, ocluidos y truncados (constatado en el EDA). Por ello, el análisis por clase debe interpretar ambos factores de forma conjunta.

3.8 Acción planificada

El punto 3 no introduce modificaciones de rebalanceo en los scripts de entrenamiento. La acción asociada se limita a la fase de evaluación: calcular y reportar AP y F1 por clase para ambos modelos, y comparar específicamente su desempeño en las clases minoritarias a la luz de la diferencia entre BCE (YOLOv8) y focal loss (RT-DETR). Esto se integra con la instrumentación de métricas descrita en la sección 2.5.

Bibliografía consultada (área 3)

1. Ultralytics. Data Augmentation using Ultralytics YOLO (copy_paste, mosaic, mixup). <https://docs.ultralytics.com/guides/yolo-data-augmentation>
 2. Balance Classes During YOLO Training Using a Weighted Dataloader (tutorial). <https://y-t-g.github.io/tutorials/yolo-class-balancing/>
 3. Ultralytics. Customizing Trainer (custom loss y trainer). <https://docs.ultralytics.com/guides/custom-trainer>
-

4. Justificación de dos integrantes y elección de versiones de los modelos

4.1 El proyecto como sistema integral

El trabajo no consiste en entrenar un único modelo de detección, sino en construir un sistema integral de extremo a extremo que abarca varias disciplinas: visión por computadora y aprendizaje profundo (dos datasets y dos arquitecturas), sistemas embebidos (despliegue en Jetson con TensorRT), robótica aérea (controlador de vuelo Pixhawk e integración en un UAV F450), redes (transmisión RTSP) e integración de software. Esta amplitud es el punto de partida para justificar la participación de dos integrantes.

4.2 Justificación académica de dos integrantes

- Multidisciplinariedad. El proyecto cruza áreas distintas (aprendizaje profundo, sistemas embebidos, robótica aérea, redes e integración) que difícilmente domina y ejecuta una sola persona dentro del tiempo de un Trabajo Terminal.
- Volumen de experimentación. Se entrenan y evalúan dos arquitecturas, en varias configuraciones, sobre dos hardware de referencia (laptop y Jetson) y dos datasets (VisDrone y UAVDT). Es un estudio comparativo riguroso, no un entrenamiento único.
- Alcance poco habitual. La mayoría de los trabajos se detienen en el entrenamiento y la medición de mAP. Este cierra el ciclo hasta el vuelo real y la georreferenciación, lo que implica un trabajo de integración hardware-software

que requiere esfuerzo en paralelo.

4.3 Propuesta de división del trabajo

La siguiente división es ilustrativa; lo esencial es evidenciar dos ejes de trabajo sustanciales que convergen en la integración.

- Integrante A, percepción y modelos: datasets, EDA, preprocesamiento, entrenamiento y comparación de YOLOv8n y RT-DETR, marco experimental, métricas y validación cruzada de dominio.
- Integrante B, sistema embebido y despliegue: seguimiento (DeepSORT y ByteTrack), despliegue en Jetson con TensorRT, transmisión RTSP, georreferenciación con Pixhawk e integración en el UAV F450.
- Trabajo conjunto: integración final, pruebas de vuelo, evaluación comparativa y documentación.

4.4 Elección de las versiones de los modelos

El hardware de despliegue determina la elección: la NVIDIA Jetson Orin Nano 8 GB, objetivo final del sistema, exige los modelos más ligeros posibles para acercarse al tiempo real.

- YOLOv8n es la variante más ligera de YOLOv8 ($n < s < m < l < x$), la CNN más adecuada para cómputo de borde.
- RT-DETR-L es la variante más ligera de RT-DETR con pesos preentrenados disponibles en Ultralytics 8.4.37 (solo se ofrecen las variantes L y X; no existe una variante nano o small oficial). Es, por tanto, el detector Transformer más apto para borde dentro de la configuración estándar.

Ambos modelos se entrenan a la misma resolución ($\text{imgsz} = 1280$), condición necesaria para una comparación justa (ver área 5) y adecuada para la detección de objetos pequeños en imagen aérea. Las cifras exactas de parámetros y FLOPs de cada modelo se reportarán junto al resto de métricas en la fase de evaluación (ver área 2).

4.5 Reconocimiento de la asimetría de tamaño

RT-DETR-L es considerablemente mayor que YOLOv8n, tanto en número de parámetros como en costo de cómputo, por lo que no se trata de una comparación entre modelos del mismo tamaño. Esta asimetría no es un defecto del diseño experimental, sino parte de lo que el estudio caracteriza: refleja el costo real de emplear un detector Transformer en cómputo de borde y sustenta la hipótesis H3 (es posible que solo YOLOv8n sostenga tiempo real en la Jetson). La comparación se plantea, por tanto, entre la variante más ligera y desplegable de cada familia, no entre modelos equiparados en tamaño.

4.6 Alternativas consideradas y descartadas

- Igualar tamaños con YOLOv8m o YOLOv8l. Contradiría el objetivo de despliegue en borde, pues modelos más pesados difícilmente alcanzarían tiempo real en la Jetson.
- RT-DETR-X. Es aún más pesado que RT-DETR-L y, por tanto, menos desplegable.
- Una variante más pequeña de RT-DETR (por ejemplo con backbone ResNet-18). No se ofrece preentrenada en Ultralytics 8.4.37; habría que entrenarla desde cero, lo que añade riesgo y rompe el criterio de comparar configuraciones estándar.

La elección final es la variante más ligera y estándar de cada familia que el despliegue permite.

Bibliografía consultada (área 4)

1. Zhao, Y. et al. DETRs Beat YOLOs on Real-time Object Detection (RT-DETR). arXiv 2304.08069. <https://arxiv.org/abs/2304.08069>
 2. Ultralytics. RT-DETR (variantes y pesos preentrenados). <https://docs.ultralytics.com/models/rtdetr/>
 3. Ultralytics. YOLOv8 (variantes n/s/m/l/x). <https://docs.ultralytics.com/models/yolov8/>
-

5. Marco de experimentación justo

5.1 Qué significa

Una comparación es justa cuando las diferencias entre YOLOv8n y RT-DETR se deben a las arquitecturas y no a condiciones dispares. No implica usar hiperparámetros idénticos, sino mantener constante lo que debe serlo y justificar lo que difiere por razones propias de cada arquitectura.

5.2 Qué se controla y qué puede diferir

Se mantiene constante en ambos modelos: dataset y splits oficiales, resolución (imgsz = 1280), semilla y determinismo (seed = 0), épocas y criterio de paro (100, patience = 20) y el protocolo de evaluación.

Difiere de forma justificada: lr0 (0.001 frente a 0.0001) y weight_decay (0.0005 frente a 0.0001), por la sensibilidad de los Transformers, y la función de pérdida (BCE frente a Focal/Varifocal), nativa de cada arquitectura (ver área 3). Cada modelo se entrena en su mejor configuración estándar.

5.3 Entrenamiento en la nube y medición de eficiencia

El entrenamiento se realiza en servicios en la nube, por su mayor capacidad y estabilidad frente al equipo local. Esto no afecta la equidad de la comparación de precisión, porque el mAP no depende del hardware donde se entrene: la justicia la garantizan los datos, la configuración, la semilla y el protocolo de evaluación, que sí se controlan.

Las métricas de eficiencia (FPS, latencia, VRAM, RAM) sí dependen del hardware y se miden aparte, en inferencia, sobre la laptop y la Jetson, con el mismo formato (TensorRT FP16) y el mismo tamaño de lote (batch = 1).

5.4 Documentación para la reproducibilidad

Se registran: versiones del entorno (Python, PyTorch, CUDA, Ultralytics), hiperparámetros completos (args.yaml por run), semilla, versión del dataset y splits, commit de git del código y protocolo de evaluación (validación frente a prueba, umbrales de IoU y confianza).

5.5 Acciones para formalizar el marco

Para dejar el marco completo y reproducible quedan pendientes estas acciones, que se ejecutan en la fase de implementación y evaluación:

- Definir y documentar el protocolo de medición de FPS en hardware fijo (laptop y Jetson, TensorRT FP16, batch = 1); se implementa dentro del script de benchmark.
- Justificar de forma explícita en la memoria las diferencias de lr0 y weight_decay entre ambos modelos.
- Registrar el identificador de commit de git asociado a cada entrenamiento.
- Evaluar en el conjunto de prueba (test), no solo en validación.

- Realizar la validación cruzada VisDrone a UAVDT, lo que requiere descargar las imágenes de UAVDT y documentar el mapeo de sus tres clases (car, truck, bus) a las de VisDrone.
-

6. Mejora de resultados

6.1 Qué es el fine-tuning y qué papel juega COCO

Entrenar un detector desde cero, con pesos aleatorios, exige enormes cantidades de datos y tiempo. En su lugar se emplea el fine-tuning (ajuste fino): se parte de un modelo ya entrenado sobre un gran dataset general y se continúa su entrenamiento sobre el dataset propio.

COCO es ese dataset general de referencia: contiene del orden de cientos de miles de imágenes y ochenta categorías cotidianas (personas, autos, etc.). Los pesos yolov8n.pt y rtdetr-l.pt que ofrece Ultralytics ya vienen entrenados en COCO, y ese conocimiento está contenido dentro del propio archivo de pesos. Es decir, no se descarga el dataset COCO: el aprendizaje adquirido en COCO viaja dentro del archivo .pt, que Ultralytics descarga automáticamente la primera vez.

Al entrenar esos pesos sobre VisDrone, el modelo aprovecha lo que ya sabe (bordes, texturas, formas básicas de los objetos) y solo lo adapta a la perspectiva aérea y a las clases de VisDrone. Por ello el proyecto ya aplica fine-tuning: cargar el archivo .pt y entrenar sobre VisDrone es, precisamente, un ajuste fino a partir de COCO.

6.2 Los backbones que ya usan los modelos

Un detector se compone, a grandes rasgos, de tres partes: el backbone, que extrae características de la imagen (bordes, texturas, formas); el cuello (neck), que combina esas características a distintas escalas; y la cabeza (head), que predice las cajas y las clases. El backbone es, en esencia, los ojos que convierten los píxeles en información útil.

- YOLOv8 utiliza CSPDarknet, un backbone convolucional diseñado para velocidad y eficiencia.
- RT-DETR-L utiliza HGNetv2, un backbone convolucional moderno y eficiente, acoplado a la parte Transformer del modelo.

Ambos son backbones recientes y optimizados. ResNet-50, en cambio, es un backbone de 2015, más pesado y menos eficiente; sustituir por él los backbones actuales no aportaría mejoras y complicaría el sistema, además de romper la comparación de configuraciones estándar (ver área 4). Por tanto, no se contempla cambiar de backbone.

6.3 Vías para mejorar el entrenamiento

- Más épocas. El entrenamiento de YOLOv8n no activó la parada temprana (patience = 20) y aún mejoraba al alcanzar las 100 épocas, lo que indica margen para entrenar más tiempo (por ejemplo, 150 a 200 épocas) y elevar el mAP.
- Ajuste de la tasa de aprendizaje y su programación (warmup y descenso tipo cosine), para refinar la convergencia, especialmente en RT-DETR.
- SAHI (Slicing Aided Hyper Inference) en la fase de inferencia: divide la imagen en recortes, detecta en cada uno y combina los resultados, lo que mejora de forma notable la detección de objetos pequeños, abundantes en VisDrone. Su costo es una inferencia más lenta, por lo que resulta más adecuada para medir el máximo de precisión que para el vuelo en tiempo real.

Las vías que finalmente se adopten se aplicarán en la fase de implementación, junto con la instrumentación de métricas (ver área 2).

6.4 Validez de los resultados actuales

El argumento de que VisDrone es un dataset especialmente difícil es válido y está respaldado por la literatura: las métricas reportadas por otros trabajos son del mismo orden (ver área 1) e, incluso, el YOLOv8n del proyecto resulta competitivo frente a ellas. Asimismo, es razonable esperar que en vuelo real, con el UAV más cerca de los objetos, la detección práctica mejore respecto a las imágenes de gran altitud de VisDrone. No obstante, este último argumento debe presentarse con matices: depende de la altitud real de operación y no sustituye a la evaluación cuantitativa, por lo que conviene respaldarlo, en su caso, con pruebas de vuelo.

Bibliografía consultada (área 6)

1. Akyon, F. C. et al. Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection (SAHI). arXiv 2202.06934. <https://arxiv.org/abs/2202.06934>
2. Lin, T.-Y. et al. Microsoft COCO: Common Objects in Context. arXiv 1405.0312. <https://arxiv.org/abs/1405.0312>
3. Ultralytics. Train (fine-tuning desde pesos preentrenados, parámetro freeze). <https://docs.ultralytics.com/modes/train/>

Anexo. Alternativas a Google Colab para el entrenamiento

A.1 Problema

Google Colab interrumpe la sesión y el entrenamiento de RT-DETR colapsa por falta de memoria (OOM) de forma reproducible en la época 25, lo que impide entrenamientos largos y estables. Se requiere una plataforma con sesiones estables, costo acorde a un presupuesto académico y compatibilidad directa con PyTorch y Ultralytics.

A.2 Comparación de opciones (2026)

Plataforma	Costo aproximado	Estabilidad	Observaciones
RunPod	RTX 4090 Alta (~0.34-0.69 USD/h; Tier 3/4); cobro por segundo A100 ~1.3-2 USD/h	(Secure Cloud)	Plantillas PyTorch listas, almacenamiento persistente, sin cortes aleatorios El más económico;
Vast.ai	50-70% más barato (RTX 4090 ~0.27 USD/h)	Media (instancias spot interrumpibles, aviso de 15 s)	conviene usar on-demand y checkpoints
Lambda Labs	A100 ~1.29-1.79 USD/h	Muy alta (99.5%+, datacenters propios)	Máxima estabilidad, mayor costo
Kaggle	Gratuito (~30 h/semana, T4/P100 16 GB)	Sesiones fijas de ~9-12 h	Adecuado para YOLOv8n; 16 GB resultan justos para RT-DETR-L a 1280

Plataforma	Costo aproximado	Estabilidad	Observaciones
Colab Pro	~10 USD/mes	Mejor que la versión gratuita, con límites de sesión	No resuelve por completo el problema

A.3 Recomendación

Se recomienda RunPod como opción principal, por su equilibrio entre costo, estabilidad y facilidad de uso:

- Para YOLOv8n (ligero), una RTX 4090 de 24 GB es suficiente.
- Para RT-DETR-L a $\text{imgsz} = 1280$ (el modelo que produce el OOM), conviene una GPU con mayor memoria (A40 de 48 GB o A100 de 40/80 GB), lo que además elimina el OOM.

Vast.ai es la alternativa más económica si se prioriza el presupuesto, usando instancias on-demand y guardando checkpoints. Kaggle, gratuito, sirve para pruebas rápidas y para YOLOv8n.

A.4 Sobre el OOM de RT-DETR en la época 25

Que el fallo ocurra siempre en la misma época sobre una A100 (que dispone de 40 a 80 GB de VRAM) sugiere agotamiento de la memoria RAM del sistema o la reclamación del recurso por parte de Colab, más que falta de memoria de GPU. En una instancia dedicada (RunPod o Vast.ai on-demand) se controla la RAM y el disco, por lo que es poco probable que se repita. Recomendaciones adicionales:

- Activar checkpointing y reanudación (Ultralytics guarda last.pt, lo que permite continuar un entrenamiento interrumpido).
- Asegurar suficiente RAM de sistema y un volumen de almacenamiento persistente.
- Ejecutar el entrenamiento en segundo plano (por ejemplo con nohup) para que una desconexión del navegador no detenga el proceso.

Bibliografía consultada (anexo)

1. Cloud GPU Rental 2026: RunPod vs Vast.ai vs Lambda Labs.
<https://www.promptquorum.com/local-llms/cloud-gpu-rental-comparison-2026>
2. GPU Cloud Pricing Comparison 2026 (Lambda, RunPod, Vast.ai).
<https://altstreet.investments/tools/gpu/gpu-price-comparison>
3. Top 12 Cloud GPU Providers for AI and Machine Learning in 2026 (RunPod).
<https://www.runpod.io/articles/guides/top-cloud-gpu-providers>
4. 7 cheapest cloud GPU providers in 2026 (Northflank).
<https://northflank.com/blog/cheapest-cloud-gpu-providers>